Manual Técnico – Visualizador de Algoritmos de Ordenamiento

1. Estructura del proyecto

- Visualizador ordenamiento
  - Main.java | Punto de entrada
- gui - Paneles
  - VentanaPrincipal.java | JFrame con CardLayout (menú, visualizador, reportes)
  - PanelMenu.java | Pantalla de bienvenida con navegación
  - PanelVisualizador.java | Panel principal: controles, gráfica, log, estadísticas
  - HiloOrdenamiento.java | Hilo que ejecuta los algoritmos en segundo plano
  - PanelReportes.java | Tabla de historial y generación de HTML
- Reportes
  - Ejecucion.java | POJO que almacena los datos de una ejecución
  - DatosSistema.java | Utilidad para obtener perfil del sistema (OSHI + Runtime)

2. Librerías externas utilizadas

Librería	Versión	Propósito
JFreeChart	1.5.5	Gráfico de barras dinámico.
OSHI Core	6.10.0	Obtención de modelo y frecuencia del procesador.
JNA	5.18.1	Acceso nativo a APIs del sistema (usado por OSHI).
SLF4J API	2.0.17	Logging (usado por OSHI).
JNA Platform	5.18.1	Extensión de JNA con clases específicas para Windows

3. Requisitos técnicos cumplidos

Manual Técnico – Visualizador de Algoritmos de Ordenamiento

- **Lenguaje:** Java 11 (o superior).
- **Interfaz gráfica:** Swing (JFrame, JPanel, JButton, JComboBox, JTextArea, JTable, etc.).
- **Estructuras de datos:** exclusivamente `int[]` (no se usan ArrayList, Vector ni colecciones).
- **Algoritmos implementados manualmente:** Bubble Sort, Shell Sort, Quick Sort (recursivo).

- **Concurrencia:** hilo propio (`extends Thread`) con `Thread.sleep()` para animación. Todas las actualizaciones de UI se realizan mediante `SwingUtilities.invokeLater()` .
- **Visualización gráfica:** JFreeChart con `BarRenderer` personalizado para colores dinámicos.
- **Perfil del sistema:** `Runtime.getRuntime()` , `System.getProperty()` y OSHI.
- **Generación de números aleatorios:** `Math.random()` .

4. Lógica general de la aplicación

La aplicación consta de tres pantallas gestionadas con `CardLayout` :

1. **Menú principal (`PanelMenu`)**: Ofrece tres botones para navegar a Visualizador, Reportes y Ayuda.
2. **Visualizador (`PanelVisualizador`)**: Es el núcleo de la práctica. Permite:
   - Cargar datos desde ingreso manual (separados por comas), desde un archivo `.txt` o de forma aleatoria (rango 1-100).
   - Seleccionar el algoritmo (Bubble Sort, Shell Sort, Quick Sort), el orden (ascendente/descendente) y la velocidad de animación (Lento/Medio/Rápido).
   - Visualizar el arreglo mediante un gráfico de barras con colores que indican el estado de cada elemento (azul=normal, amarillo=comparando, rojo=intercambiando, verde=ordenado).
   - Un área de log que muestra cada operación (comparaciones e intercambios).
   - Estadísticas actualizadas en tiempo real (comparaciones, intercambios, iteraciones).
   - Al presionar "Iniciar ordenamiento", se clona el arreglo actual, se guardan los datos para el reporte y se lanza un hilo secundario (`HiloOrdenamiento`) que ejecuta el algoritmo mientras actualiza la interfaz.
3. **Reportes (`PanelReportes`)**: Muestra una tabla con el historial de todas las ejecuciones de la sesión actual. Cada vez que se completa un ordenamiento, se genera un archivo HTML en la carpeta `reportes/` con el resumen de la ejecución (algoritmo, orden, tamaño, comparaciones, intercambios, iteraciones, tiempo, arreglo original, arreglo ordenado, velocidad y perfil del sistema).

El uso de `CardLayout` permite que los paneles no se destruyan al navegar, por lo que el hilo de ordenamiento sigue ejecutándose incluso si el usuario cambia al menú o a reportes. Al regresar al visualizador, se ve el progreso actual.

5. Explicación detallada de métodos importantes

5.1 Clase `PanelVisualizador`

Método	Descripción detallada
`crearComponentes()`	Inicializa todos los componentes Swing: campos de texto, botones, combos, área de log, etiquetas de estadísticas y el gráfico JFreeChart. Configura un `BarRenderer` personalizado que sobrescribe `getItemPaint()` para devolver el color de cada barra según el arreglo `estadoColores`. Además, activa las etiquetas de valores sobre las barras (`StandardCategoryItemLabelGenerator`).
`construirLayout()`	Organiza la interfaz usando `BorderLayout` en el panel principal. El panel superior (`panelSuperior`) tiene tres regiones: `WEST` (controles), `CENTER` (estadísticas), `EAST` (log). Los controles se colocan con `GridBagLayout` mediante el método auxiliar `agregarA()`. Cada sección se envuelve con `envolverConTitulo()` para añadir un título centrado sin usar `TitledBorder`.
`iniciarOrdenamiento()`	Valida que haya un arreglo cargado. Guarda una copia del arreglo original (`arregloOriginal`), el algoritmo seleccionado, el orden, la velocidad y el tiempo de inicio. Deshabilita todos los controles (`habilitarControles(false)`). Clona el arreglo actual (`copiaParaOrdenar`) y crea una instancia de `HiloOrdenamiento` pasándole la referencia del panel, la copia, el algoritmo, el orden y el retardo. Luego inicia el hilo.
`actualizarComparacion(int posA, int posB)`	Llama a `setColorBarra()` para marcar las posiciones `posA` y `posB` con estado `1` (amarillo). Luego llama a `refrescarGrafica()` para forzar el repintado y registra el evento en el log. Este método es llamado desde el hilo dentro de `invokeLater()`.
`actualizarIntercambio(int posA, int posB)`	Similar a `actualizarComparacion`, pero marca estado `2` (rojo).
`limpiarColores(int posA, int posB)`	Restaura a `0` (azul) las barras en las posiciones indicadas, a menos que ya estuvieran en `3` (verde). Luego llama a `refrescarGrafica()`.
`marcarOrdenado(int indice)`	Cambia el estado de la barra en `indice` a `3` (verde) y refresca la gráfica.
`refrescarGrafica()`	Llama a `actualizarGrafica(arreglo)`. Este método recarga el dataset de JFreeChart con los valores actuales del arreglo, lo que provoca que el renderer personalizado reconsulte los colores.

Método	Descripción detallada
`finalizarOrdenamiento(int[] arregloFinal, int comp, int inter, int iter)`	Calcula el tiempo total transcurrido. Convierte el arreglo original y el final a cadenas legibles. Obtiene el perfil del sistema llamando a `reportes.DatosSistema.obtenerPerfil()`. Obtiene la referencia al `PanelReportes` a través de la ventana principal. Crea un objeto `Ejecucion` con todos los datos y lo envía a `panelReportes.agregarEjecucion()`.
`aplicarEstilos()`	Establece colores de fondo personalizados para el panel principal, los subpaneles y la gráfica. Configura el fondo del `ChartPanel`, del gráfico, del área de trazado, de las líneas de cuadrícula y de la leyenda. También ajusta el color de las etiquetas de los ejes.

5.2 Clase `HiloOrdenamiento`

Método	Descripción detallada
`run()`	Primero, mediante `invokeLater()`, reinicia las estadísticas y el log, y añade el mensaje de inicio. Luego, según el algoritmo seleccionado, llama a `ejecutarBubbleSort()`, `ejecutarShellSort()` o `ejecutarQuickSort()`. Finalmente, en otro `invokeLater()`, marca todas las barras como ordenadas, llama a `finalizarOrdenamiento()` para generar el reporte, añade el mensaje de finalización y vuelve a habilitar los controles.
`ejecutarBubbleSort()`	Implementación clásica con dos bucles anidados. En cada comparación incrementa `comparaciones` y llama a `marcarComparacion()`. Si es necesario intercambiar, incrementa `intercambios`, llama a `marcarIntercambio()`, realiza el intercambio, refresca la gráfica y espera. Después de cada comparación o intercambio, limpia los colores y espera. Al final de cada pasada, marca como ordenado el último elemento.
`ejecutarShellSort()`	Utiliza una secuencia de gaps empezando por `n/2` y dividiendo entre 2 hasta llegar a 1. Para cada gap, recorre el arreglo desde `i = gap` hasta el final, y realiza una inserción dentro de la subsecuencia de paso `gap`. Compara `arreglo[j-gap]` con `valorActual` y desplaza si corresponde. Cada comparación e intercambio actualiza la UI mediante

Método	Descripción detallada
	los métodos auxiliares. No marca elementos durante la ejecución; al final, el hilo principal marca todos como verdes.
`ejecutarQuickSort(int inicio, int fin)`	Método recursivo. Si `inicio >= fin` retorna. Incrementa `iteraciones` y actualiza la UI. Llama a `particionar()` para obtener la posición final del pivote, luego marca ese pivote como ordenado. Finalmente, ordena recursivamente la sublista izquierda y derecha.
`particionar(int inicio, int fin)`	Elige el pivote como `arreglo[fin]`. Recorre `j` desde `inicio` hasta `fin-1`. Mantiene un índice `i` que delimita la zona izquierda. Si el elemento actual debe ir a la izquierda (según orden), incrementa `i` e intercambia `arreglo[i]` con `arreglo[j]`. Cada comparación e intercambio se visualiza. Al final, coloca el pivote en `i+1` e intercambia con `arreglo[fin]`. Devuelve `i+1`.
`marcarComparacion()`, `marcarIntercambio()`, `limpiarColores()`, `refrescarGrafica()`, `actualizarEstadisticasUI()`	Todos estos métodos encapsulan una llamada a `SwingUtilities.invokeLater()` que a su vez llama al método correspondiente del panel. Esto garantiza que las actualizaciones de la interfaz se ejecuten en el EDT.
`esperar()`	Llama a `Thread.sleep(retardoMs)` y captura `InterruptedException` para restaurar el estado de interrupción.

5.3 Clase `PanelReportes`

Método	Descripción detallada
`agregarEjecucion(Ejecucion e)`	Incrementa el contador de ejecuciones, añade una fila al `DefaultTableModel` de la tabla (con los datos de la ejecución) y llama a `generarArchivoHTML(e)`.
`generarArchivoHTML(Ejecucion e)`	Crea la carpeta `reportes/` si no existe. Genera un nombre de archivo único con fecha y número de ejecución. Escribe un archivo HTML simple (sin estilos complejos) que incluye: número de ejecución, fecha, tabla de resumen, arreglo original, arreglo ordenado y perfil del sistema.

5.4 Clase `DatosSistema`

Método	Descripción detallada
`obtenerPerfil()`	Construye una cadena con: `os.name` y `os.arch`; núcleos (`Runtime.availableProcessors()`); RAM total y disponible de la JVM (`Runtime.totalMemory()` y `freeMemory()`). Luego, dentro de un bloque `try-catch`, usa OSHI para obtener el modelo del procesador (`cpu.getProcessorIdentifier().getName()`), la frecuencia (`getVendorFreq() / 1_000_000`) y la RAM física total (`hal.getMemory().getTotal()`). Si ocurre una excepción, añade un mensaje de error.

6. Gestión de colores en la gráfica

- Se mantiene un arreglo `estadoColores` de tipo `int[]` del mismo tamaño que el arreglo visualizado.
- El `BarRenderer` personalizado sobrescribe `getItemPaint(int serie, int indice)`. Este método consulta `estadoColores[indice]` y devuelve:
  - `0` → azul (`new Color(50, 130, 220)`)
  - `1` → amarillo (`new Color(255, 220, 0)`)
  - `2` → rojo (`new Color(220, 50, 50)`)
  - `3` → verde (`new Color(50, 200, 50)`)
- Cada vez que se cambia un estado (comparación, intercambio, ordenado), se modifica `estadoColores` y se llama a `refrescarGrafica()`, que a su vez llama a `actualizarGrafica(arreglo)`. `actualizarGrafica` recarga el dataset de JFreeChart (`datosGrafica.clear()` + `addValue`), lo que provoca un repintado completo y el renderer vuelve a ejecutar `getItemPaint()` para cada barra.

7. Flujo de generación de reportes

1. En `PanelVisualizador.iniciarOrdenamiento()` se guardan:
   - `arregloOriginal = arreglo.clone()`
   - `algoritmoUsado`, `esAscendente`, `retardoMs`, `tiempoInicio`.
2. El hilo `HiloOrdenamiento` ejecuta el algoritmo y al finalizar llama a: `panel.finalizarOrdenamiento(arreglo, comparaciones, intercambios, iteraciones)`.
3. `finalizarOrdenamiento` calcula el tiempo total (`System.currentTimeMillis() - tiempoInicio`), convierte arreglos a texto, obtiene el perfil del sistema con `DatosSistema.obtenerPerfil()` y crea un objeto `Ejecucion`.
4. Se invoca `panelReportes.agregarEjecucion(ejecucion)`.

5. `agregarEjecucion` añade una fila a la tabla y genera un archivo HTML en la carpeta `reportes/` con el resumen.

8. Observaciones de implementación

- **Uso exclusivo de `int[]`:** No se utilizan `ArrayList`, `Vector` ni ninguna otra colección. Incluso en `PanelReportes` se evita `ArrayList` para almacenar el historial; la tabla usa `DefaultTableModel` y el contador `totalEjecuciones` es suficiente.
- **Recursión en Quick Sort:** La implementación es recursiva real, no simulada con pila, como exige el enunciado.
- **Hilos y EDT:** Todas las modificaciones de componentes Swing (cambios de texto, repintados, etc.) se realizan dentro de `SwingUtilities.invokeLater()`. Esto evita bloqueos y errores de concurrencia.
- **Persistencia entre paneles:** Gracias a `CardLayout`, los paneles no se destruyen al navegar. El hilo de ordenamiento sigue ejecutándose incluso si el visualizador no está visible, porque el hilo es independiente del EDT.
- **Formato de archivo de entrada:** Se admiten números separados por comas, espacios o saltos de línea. La función `split("[,\\s]+")` maneja estos separadores.
- **Perfil del sistema:** OSHI proporciona información detallada del hardware; si falla (por ejemplo, en sistemas operativos no soportados), se captura la excepción y se muestra un mensaje genérico. Esto evita que la aplicación se detenga.
- **Nombres de variables:** Se han utilizado nombres cortos pero significativos (`gap`, `pivote`, `indiceMenor`, `retardoMs`), facilitando la comprensión del código.

9. Compilación y ejecución

- **Requisito:** JDK 11+, Maven.
- **Compilar:** `mvn clean compile`
- **Ejecutar:** `mvn exec:java` o ejecutar la clase `Main` desde el IDE.

10. Posibles mejoras futuras (opcionales)

- Permitir pausar/detener el ordenamiento manualmente.
- Agregar más algoritmos (Selection Sort, Insertion Sort, Merge Sort).
- Guardar el historial entre sesiones (serialización o base de datos).
- Personalizar colores y fuente mediante un archivo de configuración.
- Exportar el historial a CSV o PDF.